

Project Description

Key Information

Weighting: 30%

Due Date: October 14, 2026, 11:59PM

You may choose to complete the project either in a pair or as an individual. If you choose to work as a pair, see the notes under *Working as a Pair* for further details, in particular the link to the form you must complete by the **5th of October**.

Project Overview

The [UWA handbook](#) contains critical high level information about all the University's degree courses, majors and units. This project requires you to build a knowledge graph representing a fragment of that information, and providing an OWL ontology with some basic reasoning capabilities. The knowledge graph only needs to contain the given information on courses, units, and majors - students, grades, staff etc are out of scope. The provided data (in JSON format) should provide a good overview as to what should be included. For example, you do not need to model the admission requirements to a degree course, but you should model the prerequisites for a unit!

Your ontology should run as a small standalone application, that may be implemented as a simple command line program running in a python shell, or (optionally) a small flask application.

Deliverables:

You should deliver the following elements:

1. **30%** An **OWLReady2 ontology** (possibly including pySHACL) describing the concepts in the graph, including at least: degree, unit, major, prerequisite, credits, location, outcomes. These should allow natural and useful queries over the knowledge graph, and model a number of restrictions/constraints, enabling reasoning, including:
 - A unit can't be its own prerequisite, and a prerequisite of a prerequisite is a prerequisite.
 - An outcome of a core unit is an outcome of a major.
 - A degree must have at least one major.
 - Any major of another Bachelor's degree is a major in the Bachelor of Philosophy (Honours).
 - A major of degree X is also a major of any combined degree involving degree X.
 - No major should exceed 48 credit points at level 1.

This section should be submitted as an owl file, along with the .py file used to set up the ontology, and optionally a second .py file containing any pySHACL shapes and validation functionality.

2. **25%** A knowledge graph, implemented in OWLReady2 connecting the ontology to a dataset built from the provided JSON data. You should implement a number of SPARQL queries, including:
 - Find any degree and unit pair, where the unit is a foundational unit (i.e. required in all majors of that degree)

- For each degree type (Standard/Combined/Honours), compute the average number of major options available, excluding the Bachelor of Philosophy.
- Find all majors which can be completed in Albany.
- Basic search functionality: Given a query string (eg "data analysis"), can you find the majors that have this in their outcomes?

You should submit the .py script you used to generate the knowledge graph and write the queries. You should also include any data files you used to generate the knowledge graph in your submission (these may be the provided files)

3. **10%** A simple python application, using the command line (or optionally Flask), that allows a use to browse and search the ontology/knowledge graph
 - The application should display sections of the data as requested, as well as the inferred classes and relations.
 - The application should provide a usable syntax for running queries - enabling a user to search for information.
 - The application should identify any errors in the data (according to the ontology).

The ontology, knowledge graph and application should be submitted as a zip file (*.zip). Any data files should be in a sub folder within the zip folder. If your application includes any non-standard python libraries beyond the tools listed in the Resources section of this project, you should clearly identify these in a README inside the zip folder, as well as your user manual.

4. **25%** A User Manual that explains how to use and run the application, including:
 - An overview of the schema and ontology rules/constraints.
 - Example Queries - describing how to form the queries and interpret the output
 - Instructions on how to add, update or remove data in the knowledge graph, and add rules to the ontology.

The user manual maybe presented as a pdf file, or using HTML/Markdown.

5. **10%** An individual report describing the process of building the project, including:
 - The design choices made in the project. Which options were considered and why did you make the choices you did?
 - What tools you used in the project, how effective were they, and what you would recommend people use in the future. Particularly clarify the use of any AI assistants you used.
 - An estimate of time spent on the different tasks in the project.
 - If you worked in pairs, a discussion of your contributions - see the note below.

The individual report should be a pdf file.

Resources

The basic tools that should be used when completing this project are:

- [owlready2](#) for representing the ontology and applying reasoning, including [RDFLib](#) and Python for building the knowledge graph and executing SPARQL queries.
- [pYSHACL](#) for applying and validating constraints on the graph.

- Optionally, [flask](#) for building a basic webserver and interface if you want to - a tutorial will be provided in an upcoming lab for anyone interested.

Additionally, the *handbook.zip* folder contains unit, major, and degree information in JSON format extracted from the University handbook using the beautifulsoup python library. You can use this data to build your knowledge graph. Note that the data might not be perfect - this could be reflected in your Ontology, how you process the data, or variations on the data files you end up including when building your knowledge graph. Your knowledge graph only needs to model the degrees, majors, and units with top level entries in these files.

Working as a Pair

If you choose to work with another CITS3005 Student on this assignment, you must **both** fill in this [form](#) by **Monday the 5th of October**. By choosing to opt in to working with another student on this assignment, you are both agreeing to work together and to divide the work equally.

You should both submit the application zip and User Manual, with each file clearly indicating both students who contributed to it. The Report component must be completed **individually**. Your individual reports should additionally summarise how work was divided, how effective you thought the collaboration was, and an honest assessment of the relative contribution of both team members.

Marking

The project will be marked by a rubric, which gives a rough indication of what is expected. You can see a pdf of the rubric below, or it is available on the assessment submission page on LMS.

Submission:

Your submission should include all the aspects outlined in the Deliverables section. At a top level this includes:

- A zip folder containing all files for your owlready2 python application
- Your user manual (as a html, md, or pdf document)
- Your individual report (as a pdf document)

Late Policy:

This assessment is subject to the usual UWA assignment late policy, including the 2 day penalty waiver. See details here: https://ipoint.uwa.edu.au/app/answers/detail/a_id/2711/~consequences-for-late-assignment-submission

GenAI Policy:

This is a Tier 2 assessment under the university's generative AI policy. AI tools are not required, but if you wish to, may be used to assist you with understanding the content, getting feedback, formatting, or as a coding assistant for parts of deliverable (3) and/or the data processing in (2). Any use of AI tools should be critically discussed in your individual report. You may not use AI to do the assignment for you - you should come up with your ontology/KG schema and approach yourself, structure your code yourself, understand

how to use features of owlready2 and rdflib yourself, and write your own queries. You *may not* use AI to write the User Manual or Report for you.

Any use of AI tools that is not discussed in your individual report, or which conflicts with the above guidance, will count as Academic Misconduct. All code will also be subject to standard plagiarism checks. Please see the UWA policy on Academic Misconduct for further details.